



Métodos Numéricos Computacionais

Igor dos Reis Gomes

RELATÓRIO TRABALHO 2 – DIFERENCIAÇÃO NUMÉRICA

Métodos Numéricos Computacionais

Bauru
2025

Bauru
2025
Igor dos Reis Gomes

RELATÓRIO TRABALHO 2 – DIFERENCIAÇÃO NUMÉRICA

Métodos Numéricos Computacionais

Trabalho teórico-prático apresentado como parte da composição da nota da disciplina de Métodos Numéricos Computacionais – Prof. Douglas Rodrigues.

1. OBJETIVO

O objetivo deste trabalho é aplicar todos os métodos de diferenciação numérica (Diferença Finita Progressiva, Diferença Finita Regressiva e Diferença Finita Central) para estimar a velocidade de um corpo em movimento a partir de dados discretos de tempo e posição, disponíveis no enunciado do trabalho. Além disso, é feita uma análise dos erros desses métodos.

2. EXPLICAÇÃO DO CÓDIGO

Neste trabalho, foi realizado um programa utilizando a linguagem de programação Python, visando resolver o problema proposto. Assim, o programa foi dividido em 2 arquivos, sendo esses chamados de `funcoes.py` e `igor_dos_reis_gomes_t2.py`.

- No arquivo `funcoes.py`, contém a implementação dos métodos de diferença finita e da função para cálculo dos erros absolutos.
- No arquivo `igor_dos_reis_gomes_t4.py`, contém a interface de execução do programa, foi organizado as resoluções do exercício proposto, além outras funções que tem como objetivo a geração de tabelas para visualização dos resultados e gráficos comparativos.

2.1. Diferença Finita Progressiva

O método de diferenças finitas progressivas foi implementado no seguinte trecho de código:

```
1  def df_progressiva(x, y):
2      n = len(x)
3      resultado = []
4
5      for i in range(n):
6          if i < n - 1:
7              h = x[i + 1] - x[i]
8              resultado.append((y[i + 1] - y[i]) / h)
9          else:
10             resultado.append(None)
11
12     return resultado
```

Nessa função, temos a aplicação do Método de Diferença Progressiva. A função recebe como parâmetros duas listas: os pontos de tempo x e suas respectivas posições y . Em cada iteração, é calculada a diferença entre o valor da posição atual e o próximo, dividida pela diferença de tempo correspondente, representando a velocidade aproximada naquele ponto. O valor final é adicionado à lista de resultados. Para pontos onde não existe um valor seguinte, o valor `None` é adicionado. Por fim, a lista final é retornada. Para a realização deste método, foi usada a seguinte fórmula:

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}$$

2.2. Diferença Finita Regressiva

O método de diferenças finitas regressivas foi implementado no seguinte trecho de código:

```
1  def df_regressiva(x, y):
2      n = len(x)
3      resultado = []
4
5      for i in range(n):
6          if i > 0:
7              h = x[i] - x[i - 1]
8              resultado.append((y[i] - y[i - 1]) / h)
9          else:
10             resultado.append(None)
11
12     return resultado
```

Fonte: Elaboração Própria

Nessa função, temos a aplicação do Método de Diferença Regressiva. A função recebe como parâmetros duas listas: os pontos de tempo x e suas respectivas posições y . Para cada ponto, fora o primeiro, a função calcula a variação de posição em relação ao ponto anterior, dividida pela variação de tempo,

o que resulta na velocidade aproximada naquele instante. No ponto onde não existe valor anterior, o resultado None é armazenado. A função retorna uma lista com todas as velocidades aproximadas. Para a realização deste método, foi usada a seguinte fórmula:

$$f'(x_i) = \frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}}$$

2.3. Diferença Finita Central

O método de diferenças finitas centrais foi implementado no seguinte trecho de código:

```
1  def df_central(x, y):
2      n = len(x)
3      resultado = []
4
5      for i in range(n):
6          if 0 < i < n - 1:
7              h = x[i + 1] - x[i - 1]
8              resultado.append((y[i + 1] - y[i - 1]) / h)
9          else:
10             resultado.append(None)
11
12     return resultado
```

Fonte: Elaboração Própria

Por fim, na função df_central temos a aplicação do Método de Diferença Central. A função recebe como parâmetros as listas de tempos x e posições y. Para cada ponto, a velocidade é estimada utilizando os valores de posição do ponto anterior e posterior, divididos pela diferença entre os tempos desses dois pontos. Nos extremos, como não existem valores anteriores/posteriores, a função retorna None. Ao final, a lista de velocidades centradas é devolvida. Para a realização deste método, foi usada a seguinte fórmula:

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1})}{x_{i+1} - x_{i-1}}$$

2.4. Cálculo dos Erros Absolutos

A função `calcular_erros()` calcula os erros absolutos entre as velocidades para cada método a partir dos valores de derivada fornecidos por uma função de regressão utilizando o programa Geogebra.

```
1 def calcular_erros(vp, vr, vc, derivadas_regressao):
2     erros_p = []
3     erros_r = []
4     erros_c = []
5
6     for i in range(len(derivadas_regressao)):
7         erros_r.append(f"{abs(vr[i] - derivadas_regressao[i]):.6f}" if vr[i] is not None else "-")
8         erros_c.append(f"{abs(vc[i] - derivadas_regressao[i]):.6f}" if vc[i] is not None else "-")
9         erros_p.append(f"{abs(vp[i] - derivadas_regressao[i]):.6f}" if vp[i] is not None else "-")
10
11     erros_r = [float(v) if v != "-" else None for v in erros_r]
12     erros_c = [float(v) if v != "-" else None for v in erros_c]
13     erros_p = [float(v) if v != "-" else None for v in erros_p]
14
15     return erros_p, erros_r, erros_c
```

Fonte: Elaboração Própria

Assim, é realizada a comparação entre os valores obtidos pelos métodos de diferenças finitas e os valores da derivada já calculados no Geogebra. A função recebe como parâmetros as listas de velocidades estimadas por cada método (progressiva, regressiva e central) e a lista com as derivadas da regressão. Para cada ponto, é calculado o erro absoluto entre os valores estimados e o valor da regressão. Caso o valor estimado seja None, o erro correspondente é representado com hífen. A função retorna três listas contendo os erros de cada método.

2.5. Tabela de Resultados

```
1 def imprimir_tabela_resultados(tempos, posicoes, vp, vr, vc):
2     headers = ["Tempo (s)", "Posição (m)", "Progressiva", "Regressiva", "Central", "Função Regressão"]
3     tabela = []
4
5     for i in range(len(tempos)):
6         p = f"{vp[i]:.2f}" if vp[i] is not None else "-"
7         r = f"{vr[i]:.2f}" if vr[i] is not None else "-"
8         c = f"{vc[i]:.2f}" if vc[i] is not None else "-"
9         der = f"{derivadas_regressao[i]:.2f}" if derivadas_regressao[i] is not None else "-"
10        tabela.append([tempos[i], posicoes[i], p, r, c, der])
11
12    print("\nTabela de Velocidades Calculadas:")
13    print(tabulate(tabela, headers=headers, tablefmt="fancy_grid"))
```

Fonte: Elaboração Própria

Neste trecho de código, é realizada a organização dos dados em formato de tabela para exibição no terminal. A função recebe as listas de tempos, posições, e velocidades estimadas por cada método, além das derivadas da função de regressão. Em cada linha da tabela, são exibidos os valores calculados para cada tempo, para comparação dos resultados entre os métodos. Foi utilizada a biblioteca `tabulate` para formatação da tabela no terminal.

2.6. Tabela de Erros

Assim como para a tabela de resultado, é criada uma tabela para exibir no terminal os erros de cada método.

```
1  def imprimir_tabela_erros(tempos, posicoes, vp, vr, vc):
2      headers = ["Tempo (s)", "Posição (m)", "Erro Prog.", "Erro Regr.", "Erro Cent."]
3      tabela = []
4
5      erros_p, erros_r, erros_c = funcoes.calcular_erros(vp, vr, vc, derivadas_regressao)
6
7      for i in range(len(tempos)):
8          ep = erros_p[i]
9          er = erros_r[i]
10         ec = erros_c[i]
11
12         tabela.append([
13             tempos[i], posicoes[i],
14             f"{ep:.2f}" if ep is not None else "-",
15             f"{er:.2f}" if er is not None else "-",
16             f"{ec:.2f}" if ec is not None else "-",
17         ])
18
19     print("\nTabela de Erros (Diferença para a média entre métodos):")
20     print(tabulate(tabela, headers=headers, tablefmt="fancy_grid"))
21     return erros_p, erros_r, erros_c
```

Fonte: Elaboração Própria

2.7. Geração dos Gráficos Comparativos

Nessa sessão, é mostrada a geração de dois gráficos usando a biblioteca matplotlib.

```
1 def gerar_grafico_velocidades(tempos, vp, vr, vc):
2     plt.figure(figsize=(10, 5))
3     plt.plot(tempos, vp, label='Progressiva', linestyle='--', marker='o', color="red")
4     plt.plot(tempos, vr, label='Regressiva', linestyle='-', marker='^', color="green")
5     plt.plot(tempos, vc, label='Central', linestyle='-.', marker='s', color="blue")
6     plt.plot(tempos, derivadas_regressao, label="Derivadas Regressão", linestyle=':',
7              marker='8', color="orange")
8     plt.xlabel("Tempo (s)")
9     plt.ylabel("Velocidade (m/s)")
10    plt.title("Comparação dos Métodos de Diferença Finita")
11    plt.grid(True)
12    plt.legend()
13    plt.tight_layout()
14    nome_arquivo = "grafico_velocidades.png"
15    plt.savefig(nome_arquivo)
16    plt.close()
17    print(f"\nGráfico de velocidades salvo como: {nome_arquivo}")
18
19 def gerar_grafico_erros(tempos, erros_p, erros_r, erros_c):
20     plt.figure(figsize=(10, 5))
21     plt.plot(tempos, erros_p, 'o-', label='Erro Progressiva')
22     plt.plot(tempos, erros_r, 's-', label='Erro Regressiva')
23     plt.plot(tempos, erros_c, '^-', label='Erro Central')
24     plt.xlabel("Tempo (s)")
25     plt.ylabel("Erro Absoluto (m/s)")
26     plt.title("Comparação dos Erros por Método")
27     plt.grid(True)
28     plt.legend()
29     plt.tight_layout()
30     nome_arquivo = "grafico_erros.png"
31     plt.savefig(nome_arquivo)
32     plt.close()
33     print(f"\nGráfico de erros salvo como: {nome_arquivo}")
```

Fonte: Elaboração Própria

A função `gerar_grafico_velocidades()` plota as velocidades estimadas pelos métodos progressivo, regressivo e central, junto com a derivada da função de regressão. Cada curva tem estilo e cor distintos. O gráfico é salvo como `grafico_velocidades.png`. A função `gerar_grafico_erros()` plota os erros absolutos de cada método ao longo do tempo, permitindo comparar sua precisão. O gráfico é salvo como `grafico_erros.png`.

2.8. Interface

Para pleno funcionamento do programa, na função main foi criada uma interface por linha de comando para calcular as velocidades usando os três métodos, calcular os erros absolutos e gerar os gráficos para comparação.

```
1 def main():
2     tempos = [0, 3, 5, 8, 10, 13]
3     posicoes = [0, 225, 383, 623, 742, 993]
4
5     continuar = 's'
6     while continuar == 's':
7         os.system("cls")
8         print("\n===== TRABALHO 2 - DIFERENCIAÇÃO NUMÉRICA =====")
9         print("\nEscolha uma das opções abaixo:\n")
10        print("[1] Calcular velocidades")
11        print("[2] Calcular erros")
12        print("[3] Gerar gráficos comparativo entre métodos e erros")
13        print("[4] Sair")
14        opcao = input("→ ").strip()
15
16        if opcao == '1':
17            os.system("cls")
18            vp = funcoes.df_progressiva(tempos, posicoes)
19            vr = funcoes.df_regressiva(tempos, posicoes)
20            vc = funcoes.df_central(tempos, posicoes)
21            imprimir_tabela_resultados(tempos, posicoes, vp, vr, vc)
22
23        elif opcao == '2':
24            os.system("cls")
25            vp = funcoes.df_progressiva(tempos, posicoes)
26            vr = funcoes.df_regressiva(tempos, posicoes)
27            vc = funcoes.df_central(tempos, posicoes)
28            imprimir_tabela_erros(tempos, posicoes, vp, vr, vc)
29
30        elif opcao == '3':
31            os.system("cls")
32            vp = funcoes.df_progressiva(tempos, posicoes)
33            vr = funcoes.df_regressiva(tempos, posicoes)
34            vc = funcoes.df_central(tempos, posicoes)
35            gerar_grafico_velocidades(tempos, vp, vr, vc)
36            erros_p, erros_r, erros_c = funcoes.calcular_erros(vp, vr, vc, derivadas_r
37            egressao)
38            gerar_grafico_erros(tempos, erros_p, erros_r, erros_c)
39
40        elif opcao == '4':
41            print("Saindo ... ")
42            break
43        else:
44            print("\nOpção inválida.")
45
46        continuar = input("\nDeseja continuar? (s/n) ").strip().lower()
47        if continuar != 's':
48            print("\nFinalizando o programa ... \n")
```

Fonte: Elaboração Própria

Nessa parte do código, é construída uma interface de linha de comando com um menu interativo. O usuário pode escolher entre calcular as velocidades, calcular os erros, gerar os gráficos ou sair do programa. Cada opção chama as funções responsáveis por realizar os respectivos cálculos e exibições.

3. ANÁLISE E DISCUSSÃO

Neste tópico será realizada a comparação entre os métodos de diferenciação numérica utilizados: Diferença Progressiva, Diferença Regressiva e Diferença Central. Os métodos foram aplicados sobre o mesmo conjunto de dados experimentais, permitindo uma avaliação consistente entre eles.

Tabela 1 - Conjunto de dados utilizados

Tempo (s)	0	3	5	8	10	13
Posição (m)	0	225	383	623	742	993

Além disso, foi utilizada uma função de regressão para estimar a derivada real da posição, servindo como base de comparação para os métodos numéricos. Os valores da derivada para cada ponto foram obtidos via Geogebra:

Tabela 2 - Derivadas Regressão

Valores	76.31	76.16	76.06	75.91	75.81	75.66
---------	-------	-------	-------	-------	-------	-------

Fonte: Elaboração própria

Com base nesses dados, foram calculadas as velocidades estimadas por cada método, assim como os erros absolutos em relação à derivada da regressão.

Tabela 3 - Resultados obtidos

Tempo (s)	Posição (m)	Progressiva	Regressiva	Central	Função Regressão
0	0	75.00	–	–	76.31
3	225	79.00	75.00	76.60	76.16
5	383	80.00	79.00	79.60	76.06
8	623	59.50	80.00	71.80	75.91
10	742	83.67	59.50	74.00	75.81
13	993	–	83.67	–	75.66

Fonte: Elaboração Própria

Agora, a Tabela 4 a seguir mostra os erros absolutos entre os valores entre das derivadas da regressão obtidos pelo Geogebra e os resultados obtidos pelos métodos:

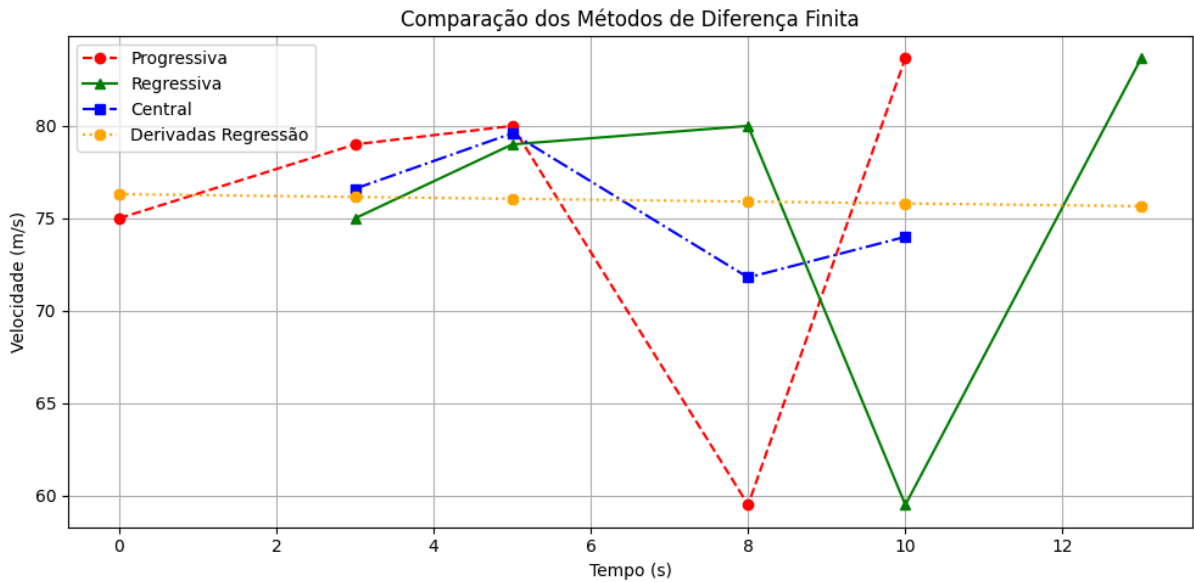
Tabela 4 - Erros obtidos

Tempo (s)	Posição (m)	Erro Prog.	Erro Regr.	Erro Cent.
0	0	1.31	–	–
3	225	2.84	1.16	0.44
5	383	3.94	2.94	3.54
8	623	16.41	4.09	4.11
10	742	7.86	16.31	1.81
13	993	–	8.01	–

Fonte: Elaboração Própria

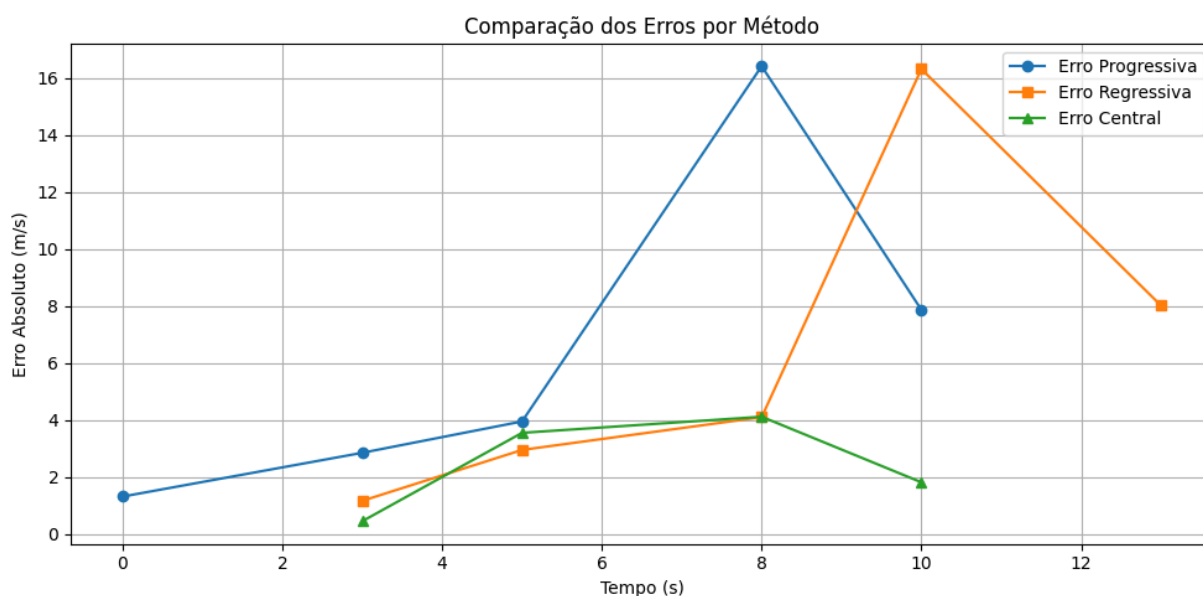
Por fim, foram gerados os seguintes gráficos para análise e visualização dos erros e dos resultados encontrados para cada método:

Figura 1 - Comparação das velocidades por método



Fonte: Elaboração Própria usando matplotlib

Figura 2 - Comparação entre erros absolutos por método



Fonte: Elaboração Própria usando matplotlib

Os gráficos e tabelas demonstram que o método de diferença finita central tende a se aproximar melhor da derivada real no interior do intervalo, enquanto os métodos progressivo e regressivo são mais eficientes nas extremidades, porém apresentam maior erro. Além disso, observa-se que a diferença central, por utilizar pontos anteriores e posteriores, tende a diminuir o erro e apresentar melhores resultados.

4. CONCLUSÃO

Com base na aplicação dos métodos de diferença progressiva, regressiva e central, e realizando as devidas comparações entre os resultados, conclui-se que todos os métodos foram implementados corretamente e produzem estimativas corretas.

O método de diferença central se mostrou o mais preciso nos pontos internos, enquanto os métodos progressivo e regressivo foram mais eficientes nas extremidades do conjunto dos dados. Os erros observados também evidenciam isso.

Portanto, todos os métodos são válidos e úteis, e sua escolha deve considerar a posição dos dados e a precisão desejada. Em problemas onde se deseja estimar derivadas em regiões internas com maior exatidão, o método central é o mais indicado. Já para estimativas nas extremidades, é aconselhado utilizar o método progressivo ou regressivo, mesmo com perda de precisão.